Politecnico di Torino
Department of Electronics and Telecommunications
Master's degree in Mechatronic Engineering

Sensors, Embedded Systems and Algorithms for Service Robotics

2025/2026

Report 1 [1]

Date, 9/11/2025

Group n° 14

Luigi Muratore –– S333098

Francesco Licci –– S338759

Davide Tocci –– S344917

Peter Furlan –– S344945

Marat Perez –– S336992

[1] GitHub Repository: https://github.com/luigimuratore/SES4SR-ros

Contents

Objective

Implement and validate an autonomous navigation controller **(Bump&Go)** on a physical TurtleBot3 robot, adapting the control software developed in simulation to interface with the real hardware's sensors and actuators correctly.

Goals

In particular, the goals are:

- understand the **hardware** components of the real robot: Turtlebot3;
- configure the **ROS2 communication** between a workstation and the robot's onboard PC via **SSH**, verifying the remote connection for node launching and data visualization;
- adapt the /scan topic subscription within the control node to utilize the qos_profile_sensor_data **Quality of Service** profile, ensuring reliable data reception from the physical LiDAR sensor;
- adapt the **LaserScan** message processing logic to determine the presence of frontal obstacles, accounting for the physical sensor's specifications;
- test the controller in a real world scenario;
- record data from all the topics using **ROSBAG** and evaluate them with **PlotJuggler**;
- **tune parameter** and refine the functions to improve the controller performance to make the program able to achieve the navigation task.

Design

Design Our ROS2 program is contained within a ROS2 package named **lab03_pkg**, which includes a node, a params file, and a launch file.

Main structural elements

Node: The controller.py file implements a **"Bump&Go"** obstacle avoidance algorithm using odometry and LiDAR data. It defines a simple logic that moves the robot forward until it detects an obstacle; when an obstacle is detected, the robot stops and turns 90° toward the clearest side before resuming forward motion. The program defines a class Controller, which inherits from Node and is initialized with the name "controller".

Parameters: The node declares and reads several parameters, which are also stored in a configuration file. These parameters can be tuned in the config.yaml file before running the node, it will override their default values, allowing for flexible configuration.
The four parameters used are:

- **max_speed** — controls the robot's forward velocity;
- **max_turn_rate** — limits the robot's rotational speed;
- **obstacle_threshold** — defines the minimum safe distance before triggering a turn;
- **is_active** — a boolean used to enable or disable the controller logic;

Publisher: A publisher is created for the **/cmd_vel** topic, which handles velocity commands. It publishes Twist messages containing linear and angular velocity components that control the robot's movement.

Subscribers: Two subscribers are implemented:

- **/odom** — provides odometry information (position and orientation). Its function (odom_callback) extracts and stores the robot's current orientation as yaw.
- **/scan** — provides LiDAR data used to detect nearby obstacles and decide when to turn. Its function (scan_callback) uses the qos_profile_sensor_data QoS profile, as recommended for real-world sensors, and is responsible for detecting obstacles.

Timer: A 10 Hz timer (0.1 s period) triggers the timer_callback function, which serves as the main control loop. It evaluates the robot's current state and publishes the appropriate Twist command.

Program Workflow

The controller logic is implemented as a simple state machine with two states: FORWARD (default) and TURN. The Flowchart is shown in Figure 1.

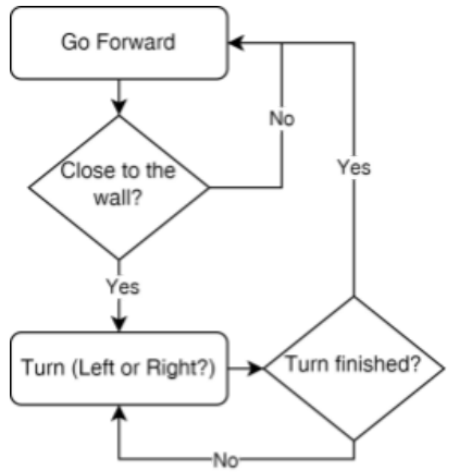


Figure 1: Flowchart of the Bump&Go algorithm.

Forward State: In this state, timer_callback continuously publishes a Twist message with a linear velocity along x equal to the configured max_speed, while all other velocity components remain zero. The transition to the TURN state is triggered by the LiDAR data processed in scan_callback. The LiDAR function isolates a 30° frontal sector by selecting the first 15 and last 15 indices from the LiDAR array (msg.ranges), representing the front field of view (FOV). It filters invalid data, like infinite, NaN, or below the minimum range, and computes the minimum valid distance. If this minimum value is smaller than the obstacle_threshold parameter we set (e.g., 0.3 m), the transition to the TURN state occurs.

Transition to Turn: When the trigger condition is met, scan_callback determines the clearer direction by comparing two 90° sectors (left and right) of the robot. The direction with the larger average distance is chosen, and the corresponding target orientation is computed by adding or subtracting 90° from the current yaw. To try to reduce odometry drift this target is adjusted using a helper function, which snaps the angle to the nearest cardinal direction $(0, \pi/2, \pi, -\pi/2)$. This angle is stored as the final target, and the node's state changes to TURN.

Turn State: In the TURN state, timer_callback stops sending forward motion (linear.x = 0.0) command and executes the rotation. It computes the shortest angular distance (diff) between the current yaw (updated from /odom) and the target yaw. A simple proportional controller (2.0 * diff) generates the rotational speed, which is clamped by configured max_turn_rate to avoid excessive rotation speed. During the turn, the function continuously checks if the rotation is complete by comparing abs(diff) to a tolerance (0.04 rad). The robot keeps rotating until the orientation error falls below this threshold.

Transition to Forward: When the turn is complete, the state transitions back to FORWARD. A short timer ensures the robot moves forward for a minimum duration before processing new obstacle detections. This prevents immediate retriggering of the TURN state. Finally, a Twist message is published with forward velocity set to max_speed and zero angular velocity to resume motion.

RViz

To monitor and debug the controller's behavior in real time, we used RViz2. For exmaple by visualizing the /scan topic, we confirmed that the LiDAR correctly detected obstacles and that the robot reacted as expected.

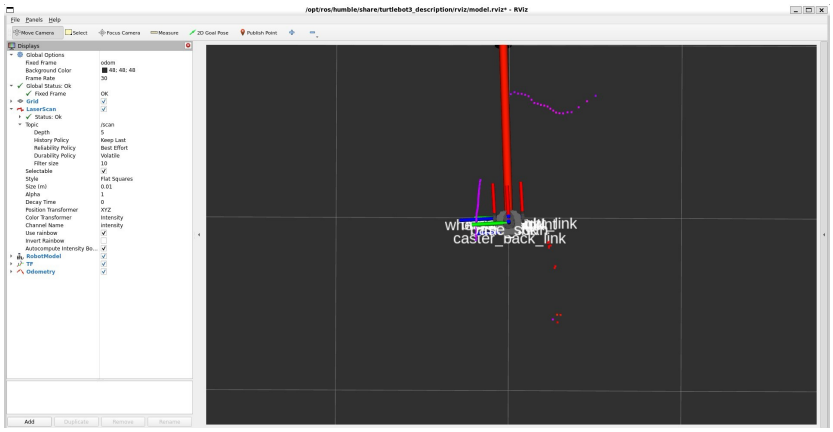


Figure 2: RViz2 visualization showing the LiDAR scan data.

Rosbag and PlotJuggler

We also recorded all topics using rosbags during tests. These recordings allowed us to visualize and analyze plots using PlotJuggler. This helped us evaluate the robot's speed, turning rates, and obstacle distances over time, providing insights for tuning.

Launch file

To run all components together, we created the lab02_launch.py launch file. This file automates the startup process by:

- launching the controller node with its parameter file;
- launching RViz2 to visualize the LiDAR scan.

Experiment

Problems

In the first attempt to make the robot move, we encountered two main issues:

- the robot did not move forward;
- the robot did not turn properly.

After checking the code, we solved the first problem by reducing the forward velocity from 0.23 m/s to 0.15 m/s, as the original speed was the the maximum speed of the robot and caused instability.
To solve the second issue, we implemented feedback control based on odometry. In particular, the /odom topic continuously updates the variables yaw and last_odom_time, which represent the robot's angular orientation (yaw angle) and the timestamp of the last odometry message received.
Firstly, we used this information to verify that the odometry data was not stale, ensuring that the control logic relied only on fresh sensor readings. As described earlier, the controller operates with two main

states: FORWARD and TURN. In the FORWARD state, the node publishes a constant forward velocity. When the scan_callback detects an obstacle within the threshold distance, it triggers a transition to the TURN state by setting a yaw target of $+90°$ or $-90°$, depending on which direction is clearer. Here we implemented the feedback control helper function that checks whether the turn is complete — that is, when the angular difference between the current yaw and the target yaw falls within the defined tolerance.

Tests

After implementing these improvements, we verified that the robot moved correctly on the test table. Once confirmed, we proceeded to test it in the real environment we had prepared.
Below is a picture of the map where the robot was tested:

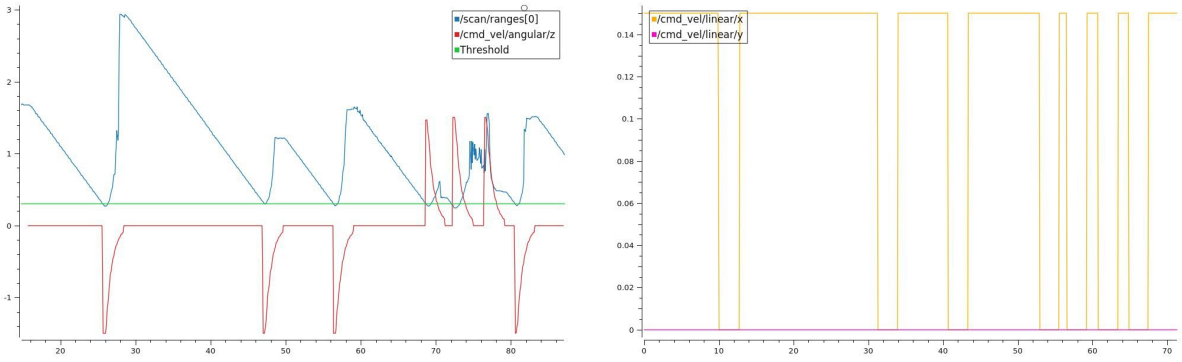
Figure 3: Map where the robot has been tested.

During the first test on the map, the robot failed to navigate correctly because the distance threshold parameter was set too high, causing it to stop too early before performing the turn. By analyzing the minimum range value from the LiDAR sensor, we selected a new threshold value smaller than the previous one but still above the sensor's minimum detection distance, ensuring the robot could safely approach the walls and turn without issues.
After a few iterations of tuning this threshold, the robot successfully navigated the entire map, avoiding obstacles and completing the task smoothly.

Results and Discussion

Plots evaluation

The primary goal was to validate the bump&go code, produced in lab02 and later modified, on a physical robot.
The controller's behavior and the resulting odometry are shown in the figure 4 and figure 5.



(a) LiDAR scan data.

(b) Linear velocity over time.

Figure 4: Robot test results.

The plots show **five critical time series** from the rosbag recording:

- the blue line (/scan/ranges[0]) represents the LIDAR center laser beam measurement, directly ahead of the robot;
- the red line (/cmd_vel/angular/z) represents the angular velocity command sent to the robot around axis z;
- the green line is just a constant with value y = 0.3 representing the distance threshold we set;
- the orange line (/cmd_vel/linear/x) and the pink one (/cmd_vel/linear/y) represent the linear velocity commands along the x and y axis respectively.

Taking in mind the controller behavior, we said that if the LiDAR sensor detects something closer than 0.3m ahead of him it turns by 90° to the direction with more free space.
The robot never moves forward and rotates at the same time.
That can be seen from the plots, and confirms the correct behavior of the controller.
As soon as the blue line crosses the green one, according to the frequency of the timer function, the angular velocity changes from zero to its max value in order to make the robot turn by 90°. In the meantime the linear velocity on the x axis is set to zero. The sign of the angular velocity depends on the direction the robot turns.
Note that: the /cmd_vel/ topic is expressed wrt to the robot's own reference frame and not to the global one. Therefore, since the robot can move only forward, the value of the linear velocity on the y axis is always zero. It is not /scan/ranges[0] that triggers the turn command in the controller, but the minimum between the first and last measurements, but we chose to display just one of those to have a clearer graph.

Robot path

Figure 5 shows the robot perceived x-y path.

It can be clearly seen that the path is not perfectly equal to the one performed in simulation.

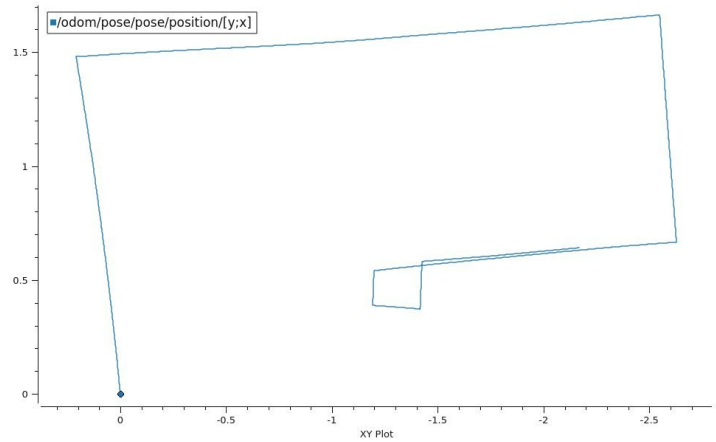


Figure 5: Odometry plot

The lines are not straight and the turns are not perfectly 90° angles.
These errors are caused by physical factors, primarily wheel slippage and secondary to sensor noise.

Conclusion

The implementation of the Bump&Go controller on the physical TurtleBot3 successfully demonstrated the transition from simulation to a real-world robotic platform. By adapting the control node to interface with actual hardware components, the robot was able to autonomously navigate the environment and detect obstacles through its LiDAR sensor. Through parameter tuning the controller achieved stable and reliable behavior.

Although the real-world trajectory showed deviations from the simulated path—mainly due to sensor inaccuracies and mechanical gap—the overall objective was achieved: the robot performed consistent obstacle avoidance and completed the navigation task autonomously.